

MédiGuide

Cahier des charges — Analyse et conception

1. Présentation du projet

MédiGuide est une application web permettant aux citoyens du gouvernorat de Jendouba de localiser rapidement des professionnels de santé (médecins toutes spécialités, pharmacies et parapharmacies) à proximité de leur position géographique. La plateforme centralise les informations pratiques de chaque praticien ou établissement (adresse, horaires, contact, spécialité), permet de consulter les avis d'autres utilisateurs et de prendre rendez-vous en ligne.

1.1 Contexte

Dans le gouvernorat de Jendouba, l'information sur les professionnels de santé disponibles (spécialité, adresse, horaires) reste dispersée et difficile d'accès pour le citoyen, qui doit souvent se déplacer ou téléphoner pour connaître une disponibilité. MédiGuide répond à ce besoin en regroupant ces informations sur une plateforme unique, accessible depuis un ordinateur ou un smartphone.

1.2 Objectifs

- Permettre à tout visiteur de rechercher un professionnel de santé ou un établissement par spécialité, ville ou proximité géographique.
- Centraliser les informations pratiques (adresse, horaires, contact) et les avis des patients.
- Permettre la prise de rendez-vous en ligne, avec confirmation automatique par email.
- Offrir aux professionnels un espace pour gérer leur profil, leurs disponibilités et leurs rendez-vous.
- Donner à l'administrateur les moyens de valider, modérer et superviser la plateforme.

1.3 Technologies utilisées

- **Frontend** : React 18 + Vite + Leaflet.js + React Query + Tailwind CSS
- **Backend** : Node.js + Express + JWT (access court + refresh rotatif) + Nodemailer
- **Base de données** : PostgreSQL 15 + PostGIS 3 (geography(Point,4326), index GiST, ST_DWithin/ST_Distance, bouton « Autour de moi »)
- **Cache & sessions** : Redis (cache recherche géo + sessions JWT)
- **Géolocalisation** : OpenStreetMap (Nominatim + Leaflet.js)

2. Diagramme de cas d'utilisation

Figure 1 : Diagramme de cas d'utilisation – MédiGuide

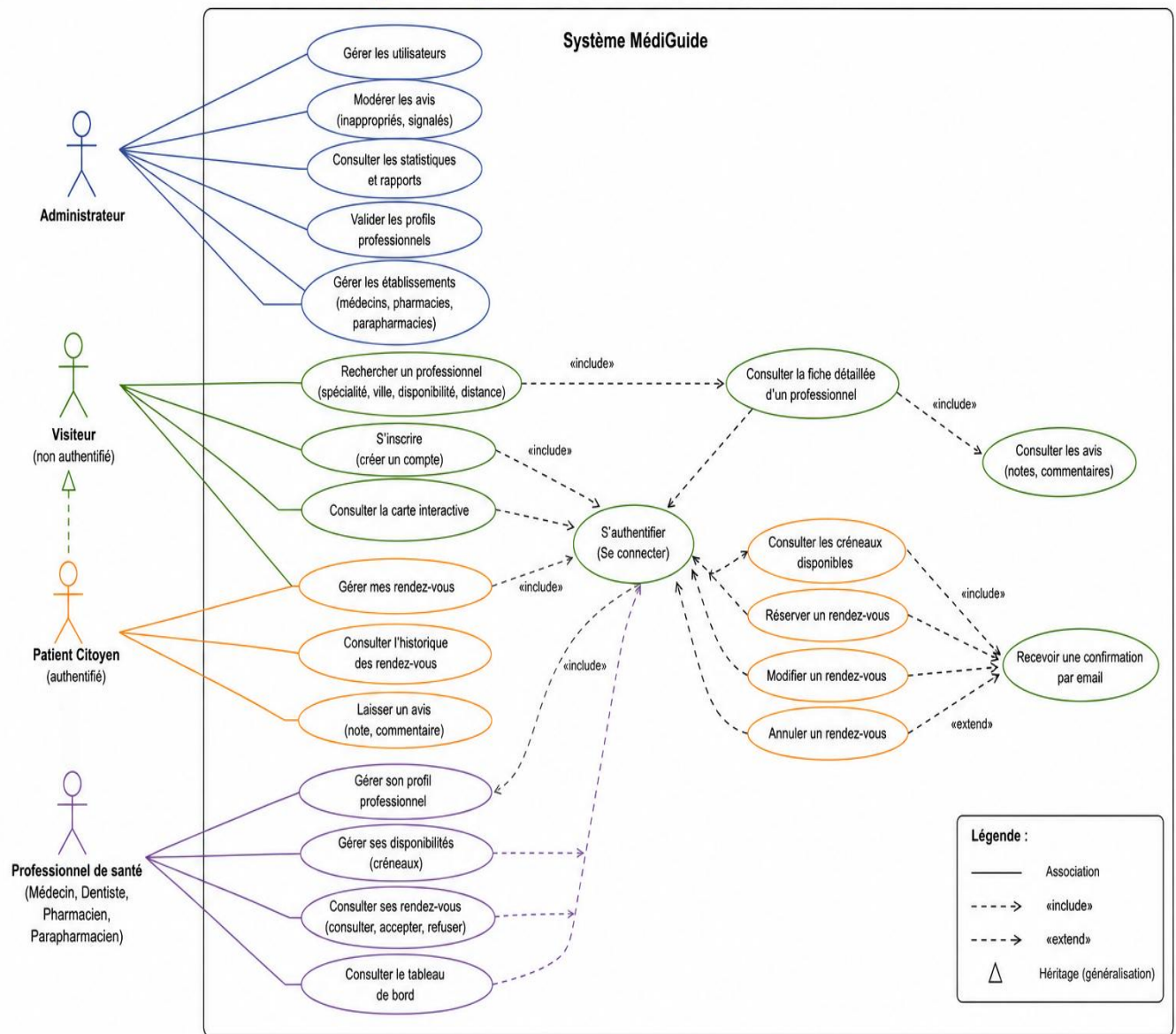


Figure 1 — Diagramme de cas d'utilisation détaillé de MédiGuide

2.1 Acteurs

Acteur	Description
Visiteur	Recherche un professionnel, consulte les fiches détaillées, les avis et la carte interactive ; peut s'inscrire.
Patient citoyen	Hérite des droits du Visiteur ; gère ses rendez-vous, consulte son historique et laisse des avis.
Professionnel de santé	Médecin, pharmacien ou parapharmacien référencé ; gère son profil, ses disponibilités et ses rendez-vous.
Administrateur	Valide les profils, gère les établissements et les utilisateurs, modère les avis, consulte les statistiques.

2.2 Cas d'utilisation — Visiteur (non authentifié)

Cas d'utilisation	Description / Relations
Rechercher un professionnel	Spécialité, ville, disponibilité, distance — inclut Consulter la fiche détaillée, qui inclut Consulter les avis
S'inscrire	Créer un compte — inclut S'authentifier
Consulter la carte interactive	Affichage des professionnels et établissements à proximité
Consulter la fiche détaillée d'un professionnel	Adresse, horaires, contact, spécialité, avis

2.3 Cas d'utilisation — Patient citoyen (authentifié, hérite du Visiteur)

Cas d'utilisation	Description / Relations
Gérer mes rendez-vous	Réserver, modifier, annuler — inclut S'authentifier ; l'annulation étend Recevoir une confirmation par email
Consulter l'historique des rendez-vous	Liste des rendez-vous passés et à venir
Laisser un avis	Note et commentaire sur un professionnel

2.4 Cas d'utilisation — Professionnel de santé

Cas d'utilisation	Description / Relations
Gérer son profil professionnel	Informations, photo, horaires
Gérer ses disponibilités	Ouverture/fermeture de créneaux
Consulter ses rendez-vous	Consulter, accepter, refuser
Consulter le tableau de bord	Rendez-vous à venir, avis reçus, activité de la fiche

2.5 Cas d'utilisation — Administrateur

Cas d'utilisation	Description / Relations
Gérer les utilisateurs	Activer, désactiver, supprimer un compte
Modérer les avis	Avis inappropriés ou signalés
Consulter les statistiques et rapports	Recherches, avis, rendez-vous
Valider les profils professionnels	Validation avant publication de la fiche
Gérer les établissements	Médecins, pharmacies, parapharmacies

3. Besoins fonctionnels

Le système MédiGuide doit permettre de :

- Localiser les professionnels de santé à proximité de la position de l'utilisateur (PostGIS + Geolocation API).
- Rechercher un professionnel par spécialité, ville ou distance, avec recherche full-text et floue (extension pg_trgm sur nom, spécialité, ville) combinée aux filtres.
- Afficher la liste des médecins, pharmacies et parapharmacies.
- Consulter la fiche détaillée d'un professionnel, incluant : adresse, horaires, téléphone, spécialité, géolocalisation, langues parlées, secteur conventionné (RPPS), et disponibilité de la téléconsultation (oui/non), note moyenne et avis des patients.
- Afficher les professionnels sur une carte interactive.
- Créer un compte utilisateur et se connecter à la plateforme.
- Gérer le profil de l'utilisateur.
- Consulter les avis et les notes des professionnels.
- Laisser un avis après une consultation, avec un **système de notation 1 à 5 étoiles** ; la moyenne est calculée côté SQL (fonction AVG) et affichée sur la fiche, la carte et la liste. L'avis n'est possible qu'après un rendez-vous confirmé.
- Consulter les créneaux disponibles d'un professionnel.
- Réserver, modifier ou annuler un rendez-vous en ligne (vérification de la disponibilité en base avant enregistrement).
- Consulter l'historique des rendez-vous.
- Recevoir une confirmation de rendez-vous par e-mail.
- Permettre aux professionnels de gérer leur profil, leurs disponibilités, et de consulter/accepter/refuser leurs rendez-vous.
- Permettre aux administrateurs de gérer les utilisateurs, valider les profils des professionnels, gérer les établissements et modérer les avis.
- Permettre aux administrateurs de consulter des **statistiques concrètes** : nombre de recherches par critère, rendez-vous par jour et par statut, note moyenne par spécialité, nombre d'avis modérés/signalés — via un endpoint agrégé SQL.

4. Besoins non fonctionnels

Le système MédiGuide doit respecter les exigences suivantes :

- **Performance** : les recherches doivent être rapides avec un temps de réponse inférieur à 3 secondes.
- **Disponibilité** : l'application doit être accessible 24h/24 et 7j/7.
- **Sécurité** : les données personnelles doivent être protégées et les mots de passe chiffrés, validation stricte des entrées (Zod), limitation du débit (express-rate-limit), CORS restreint, protection CSRF, JWT access court + refresh token rotatif, audit des dépendances (npm audit) en CI.
- **Authentification** : seuls les utilisateurs autorisés peuvent accéder aux fonctionnalités protégées.
- **Confidentialité** : les informations des utilisateurs doivent être traitées de manière confidentielle.
- **Ergonomie** : l'interface doit être simple, intuitive et facile à utiliser.
- **Compatibilité** : l'application doit fonctionner sur les principaux navigateurs web (Chrome, Firefox, Edge, Safari).
- **Responsive Design** : l'interface doit s'adapter aux ordinateurs, tablettes et smartphones.
- **Fiabilité** : le système doit garantir l'intégrité et la cohérence des données.

- **Évolutivité** : l'application doit pouvoir intégrer facilement de nouvelles fonctionnalités et de nouveaux types d'établissements.
- **Maintenabilité** : le code doit être structuré, documenté et facile à maintenir.
- **Scalabilité** : le système doit supporter un nombre croissant d'utilisateurs et de professionnels.
- **Précision de la géolocalisation** : les résultats doivent être basés sur la position réelle de l'utilisateur.
- **Sauvegarde** : les données doivent être sauvegardées régulièrement afin d'éviter toute perte d'information.
- **Accessibilité** : l'application doit être utilisable par un large public, y compris les personnes ayant des besoins spécifiques.

5. Diagramme de classes

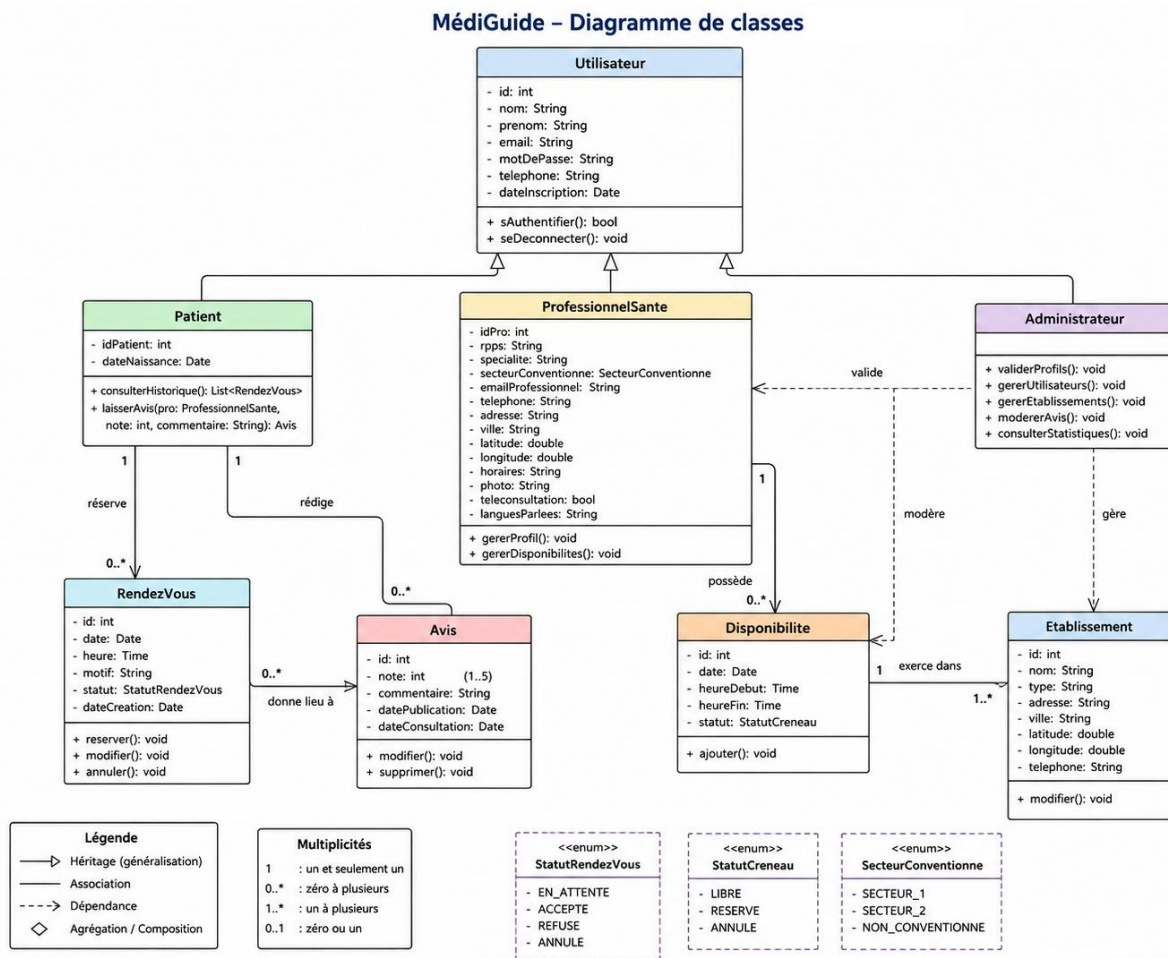


Figure 2 — Diagramme de classes de MédiGuide

5.1 Description des classes

- **Utilisateur** : classe de base représentant tout compte de la plateforme. Elle contient les informations communes (**id**, **nom**, **prénom**, **e-mail**, **mot de passe**, **téléphone**, **date d'inscription**) et fournit les fonctionnalités d'authentification (connexion et déconnexion). Elle est spécialisée en **Patient**, **ProfessionnelSante** et **Administrateur**.
- **Patient** : hérite de la classe **Utilisateur** et ajoute la **date de naissance**. Il peut réserver des **RendezVous**, consulter son historique et rédiger des **Avis** accompagnés d'une **note comprise entre 1 et 5 étoiles**.
- **ProfessionnelSante** : hérite de la classe **Utilisateur** et représente un médecin, un dentiste, un pharmacien ou un parapharmacien. Il possède notamment la **spécialité**, l'**adresse**, les **coordonnées GPS**, les **horaires**, le **numéro RPPS**, le **secteur conventionné**, les **langues parlées**, l'option **téléconsultation**, ainsi que ses coordonnées professionnelles. Il reçoit des **RendezVous**, gère ses **Disponibilite** et exerce dans un ou plusieurs **Etablissement**.
- **Administrateur** : hérite de la classe **Utilisateur**. Il valide les profils des professionnels de santé, gère les utilisateurs et les établissements, modère les avis et consulte les statistiques de la plateforme.
- **Etablissement** : représente une clinique, une pharmacie ou une parapharmacie. Il regroupe les informations relatives au **nom**, au **type**, à l'**adresse**, aux **coordonnées GPS** et au **téléphone**. Un établissement peut accueillir un ou plusieurs professionnels de santé.
- **Disponibilite** : représente un créneau proposé par un **ProfessionnelSante**. Elle contient la **date**, l'**heure de début**, l'**heure de fin** ainsi qu'un **statut** (*Libre*, *Réservé* ou *Annulé*).
- **RendezVous** : représente un rendez-vous entre un **Patient** et un **ProfessionnelSante**. Il contient la **date**, l'**heure**, le **motif**, le **statut** (*En attente*, *Accepté*, *Refusé* ou *Annulé*) et peut être associé à un **Avis** après la consultation.
- **Avis** : représente l'évaluation d'un **ProfessionnelSante** par un **Patient** à la suite d'un **RendezVous**. Il comprend une **note comprise entre 1 et 5**, un **commentaire**, la **date de publication** ainsi que la **date de consultation**.

6. Diagramme de séquence

Figure 3 — Diagrammes de séquence des principaux scénarios de MédiGuide (6.1 à 6.10)

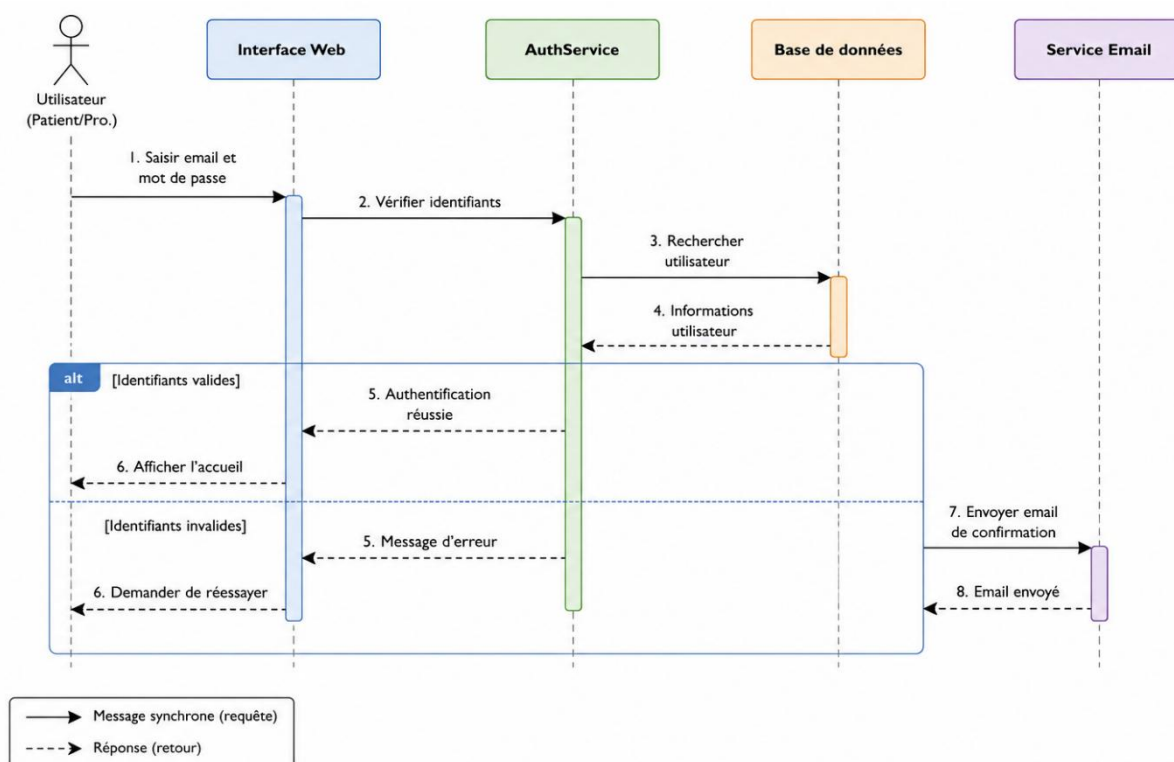


Diagramme de séquence 6.1 — Authentification (Se connecter)

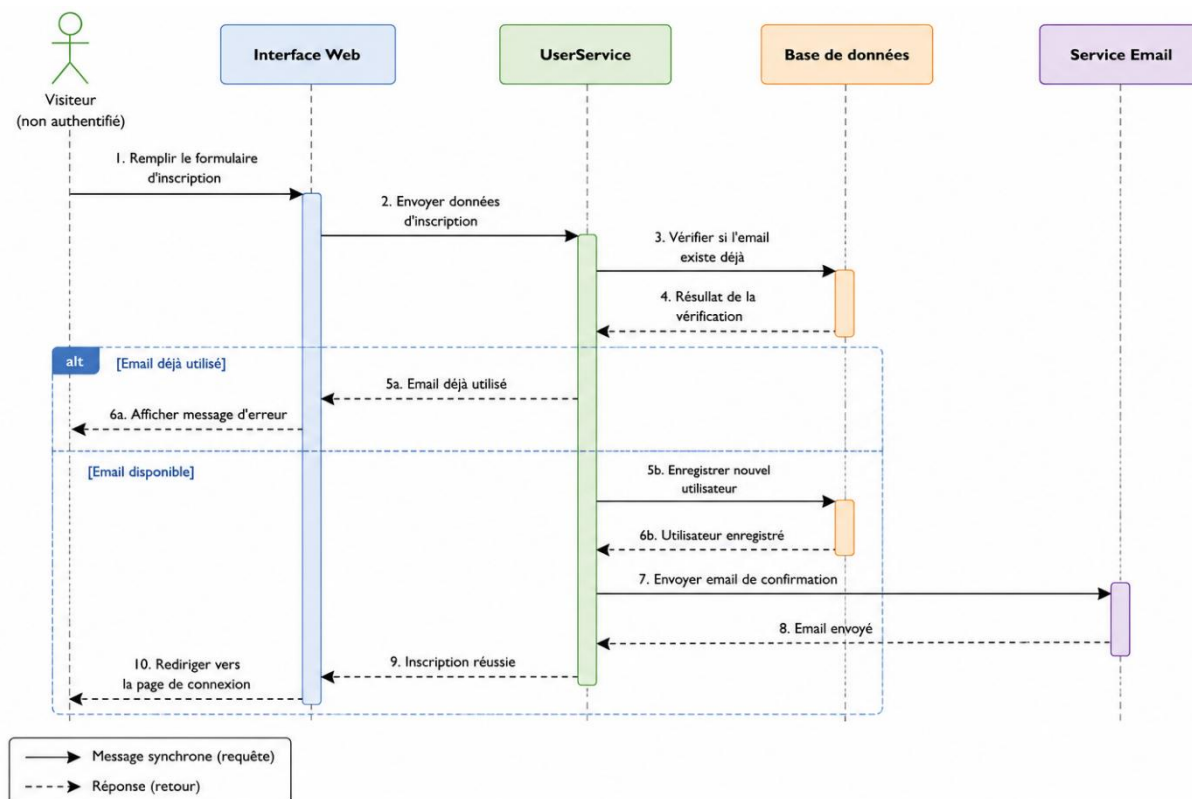


Diagramme de séquence 6.2 — Inscription (Créer un compte)

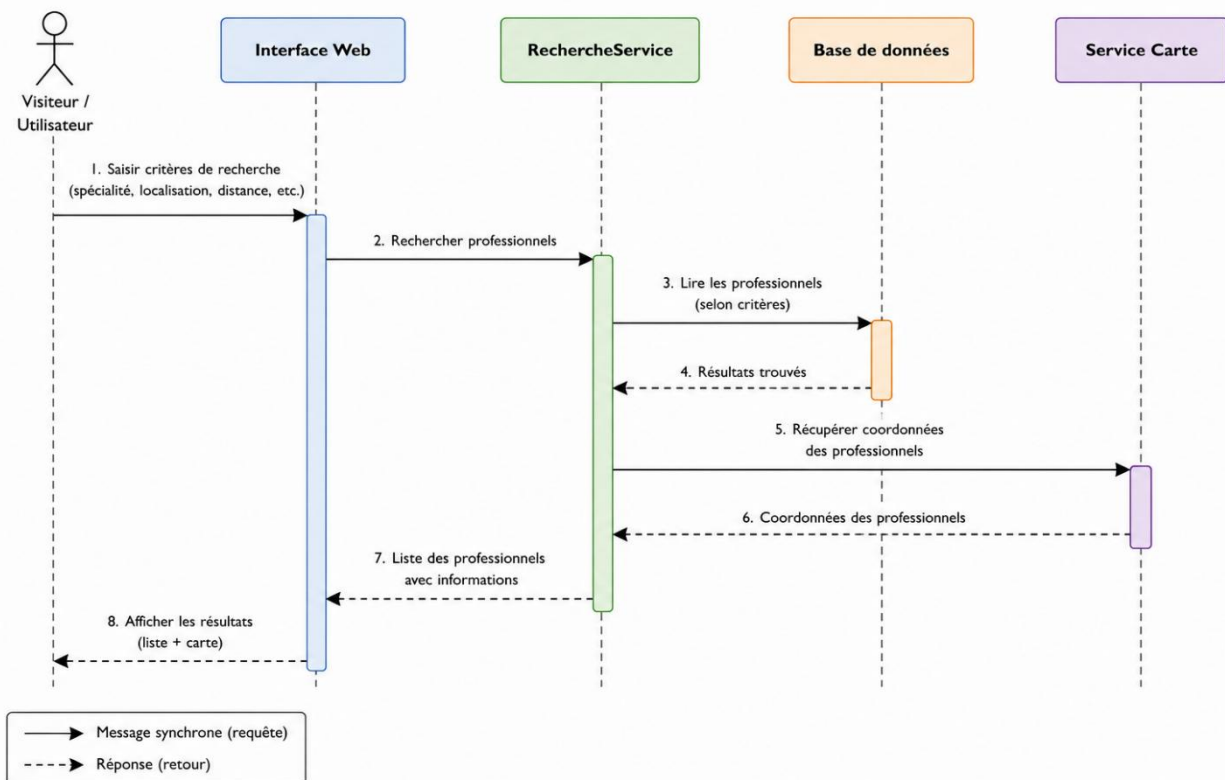


Diagramme de séquence 6.3 — Recherche d'un professionnel

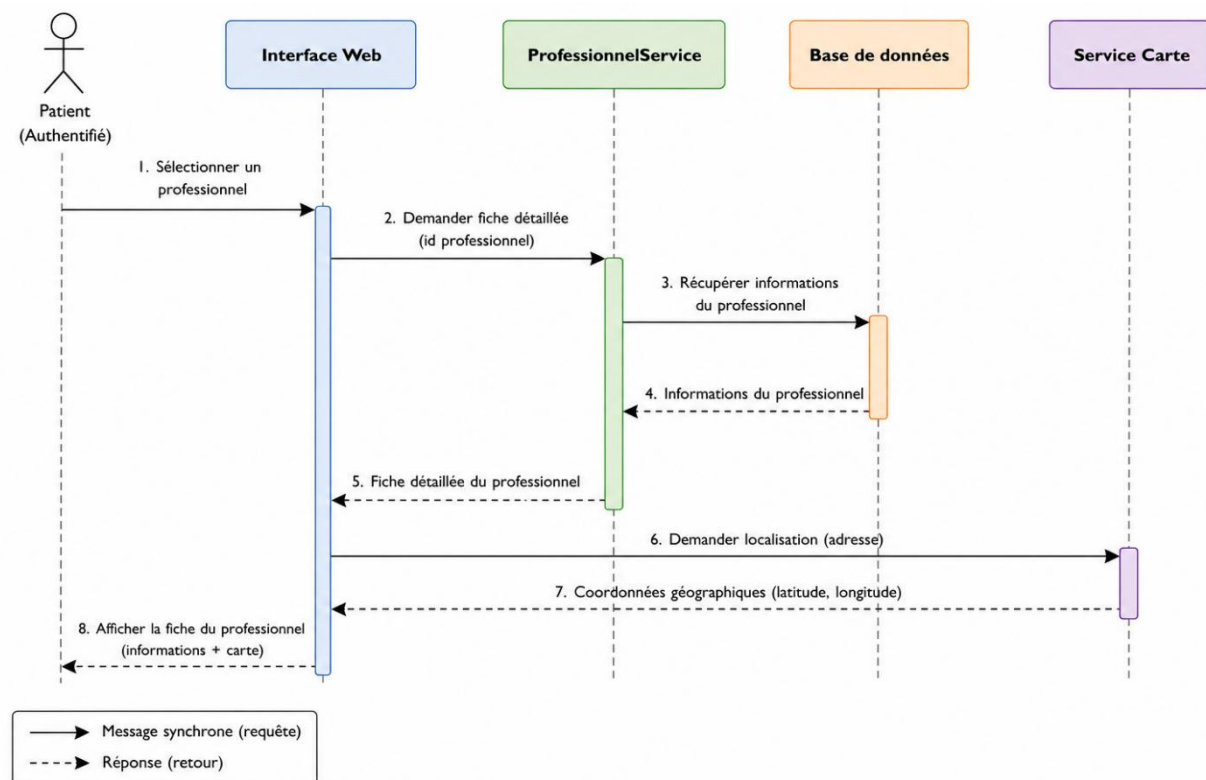


Diagramme de séquence 6.4 — Consultation de la fiche d'un professionnel

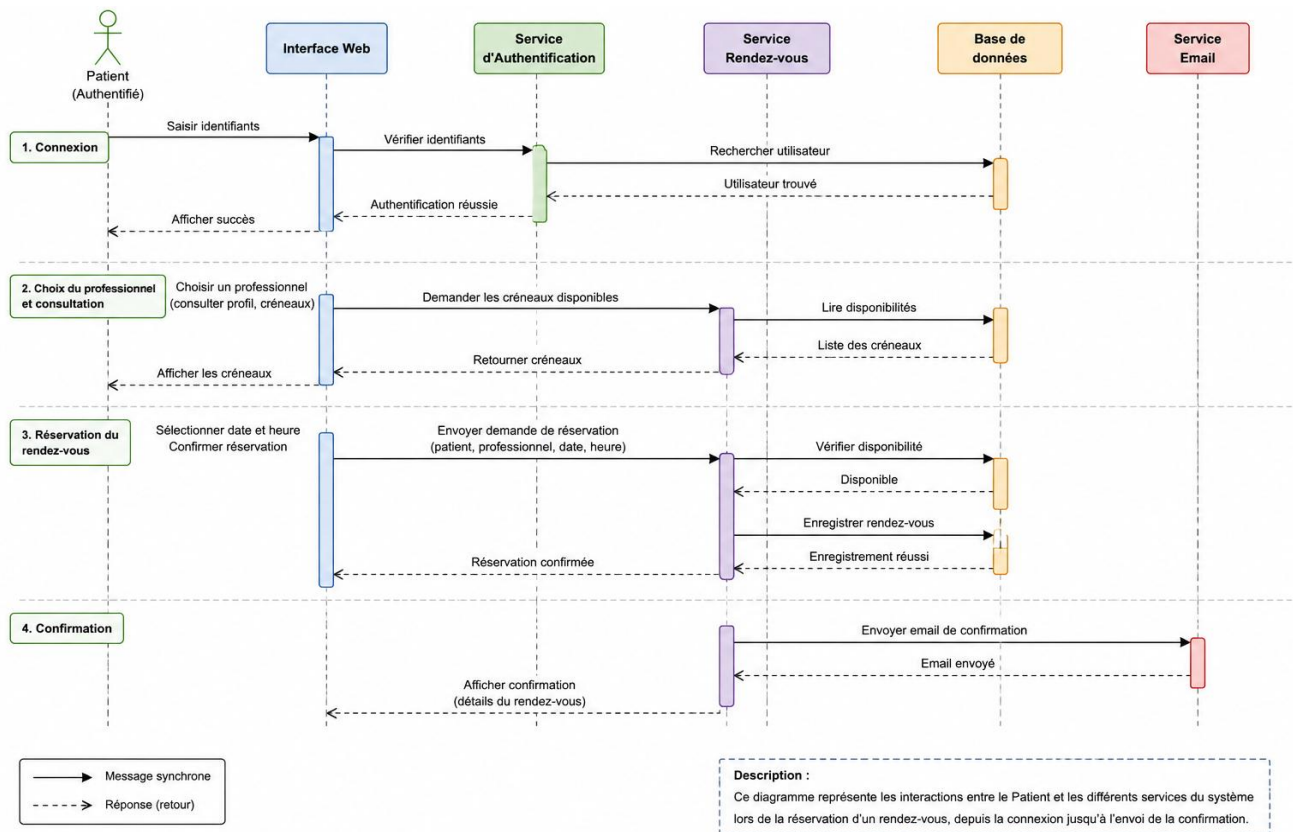


Diagramme de séquence 6.5 — Réservation d'un rendez-vous (vue détaillée)

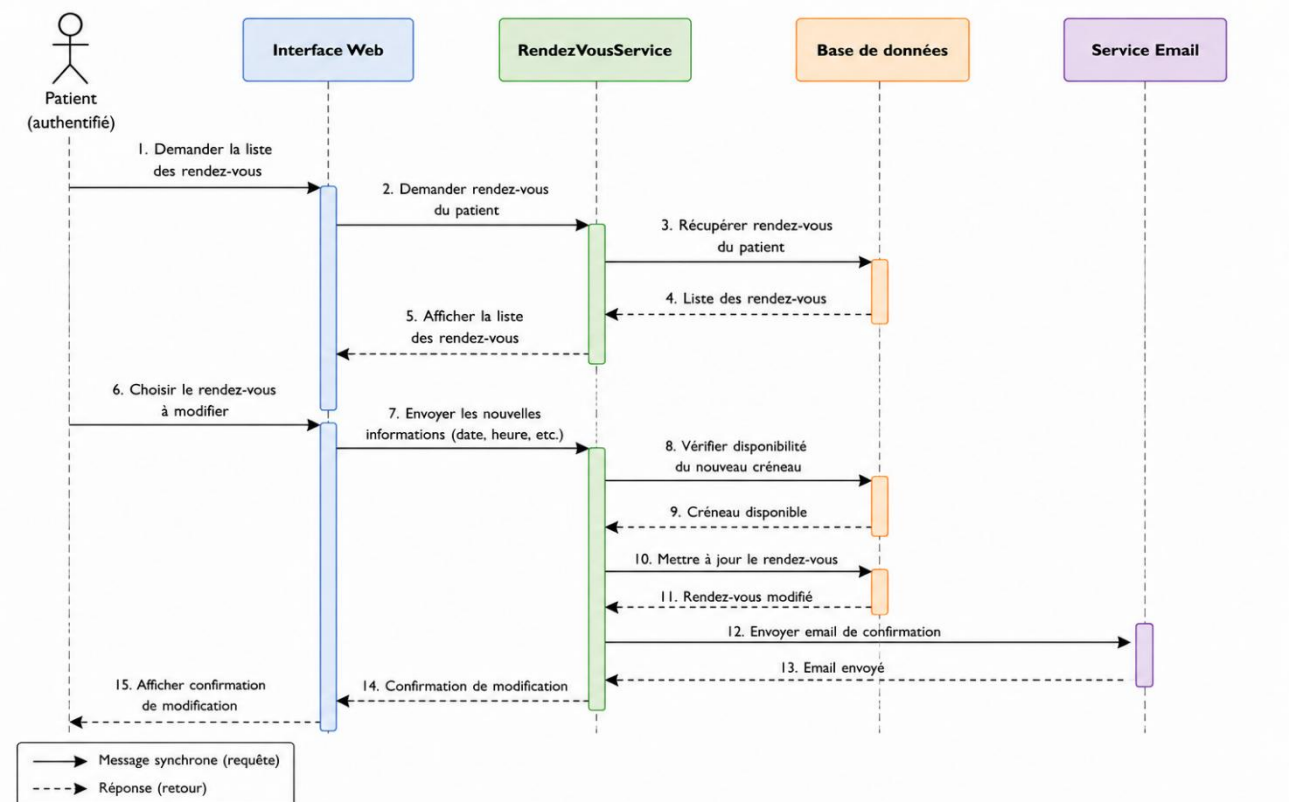


Diagramme de séquence 6.6 — Modification d'un rendez-vous

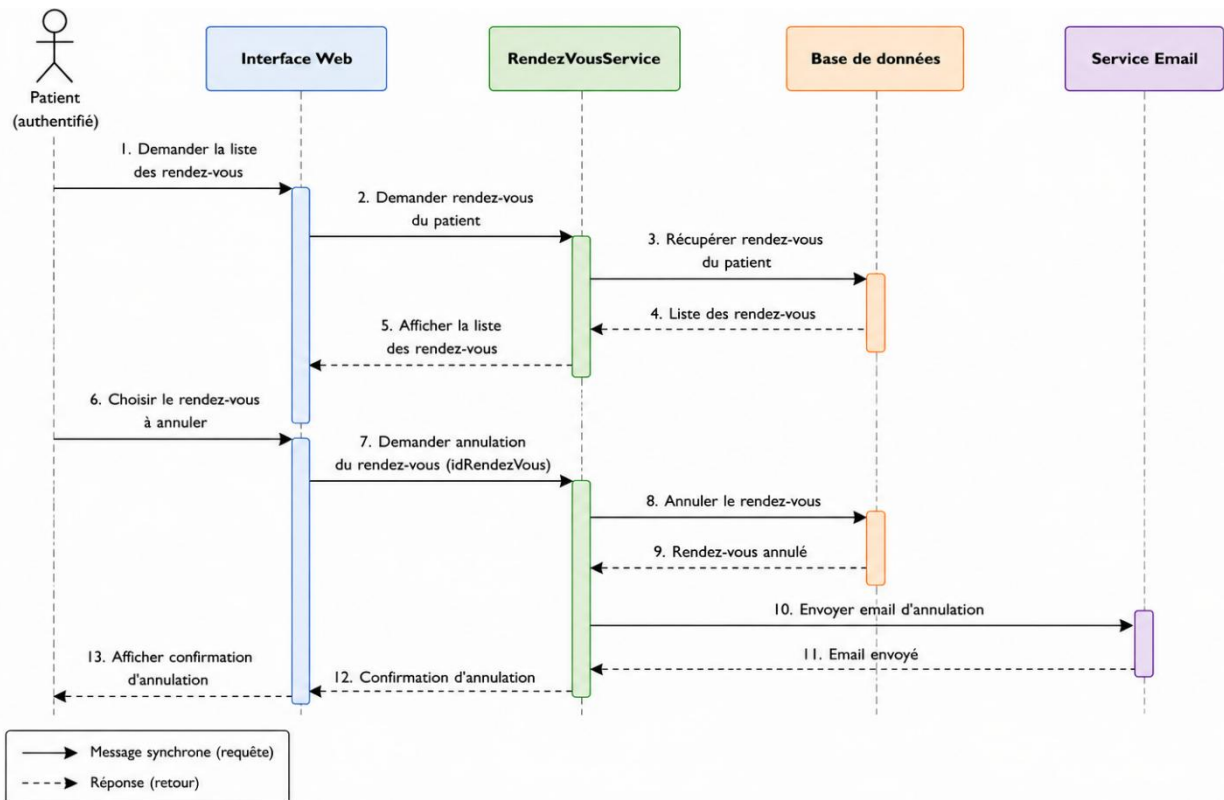


Diagramme de séquence 6.7 — Annulation d'un rendez-vous

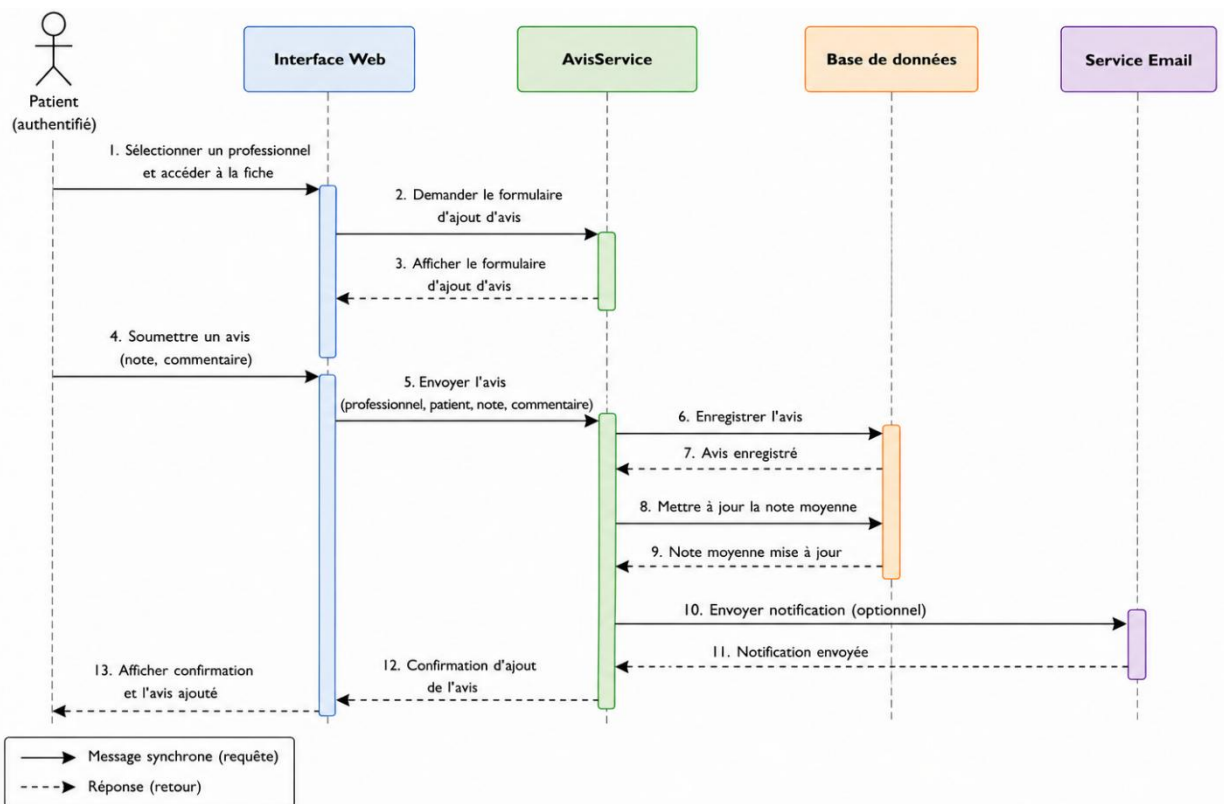


Diagramme de séquence 6.8 — Ajout d'un avis

Diagramme de séquence – Gestion des disponibilités (Professionnel de santé)

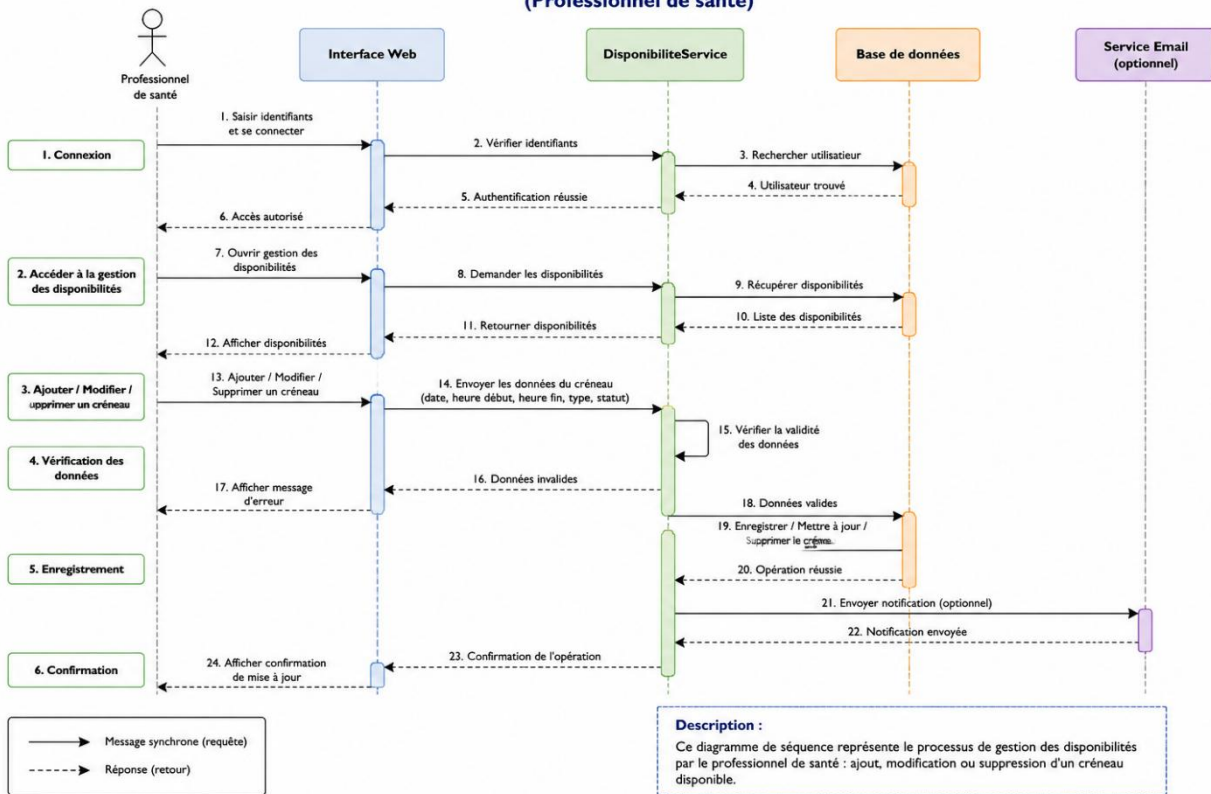


Diagramme de séquence 6.9 — Gestion des disponibilités

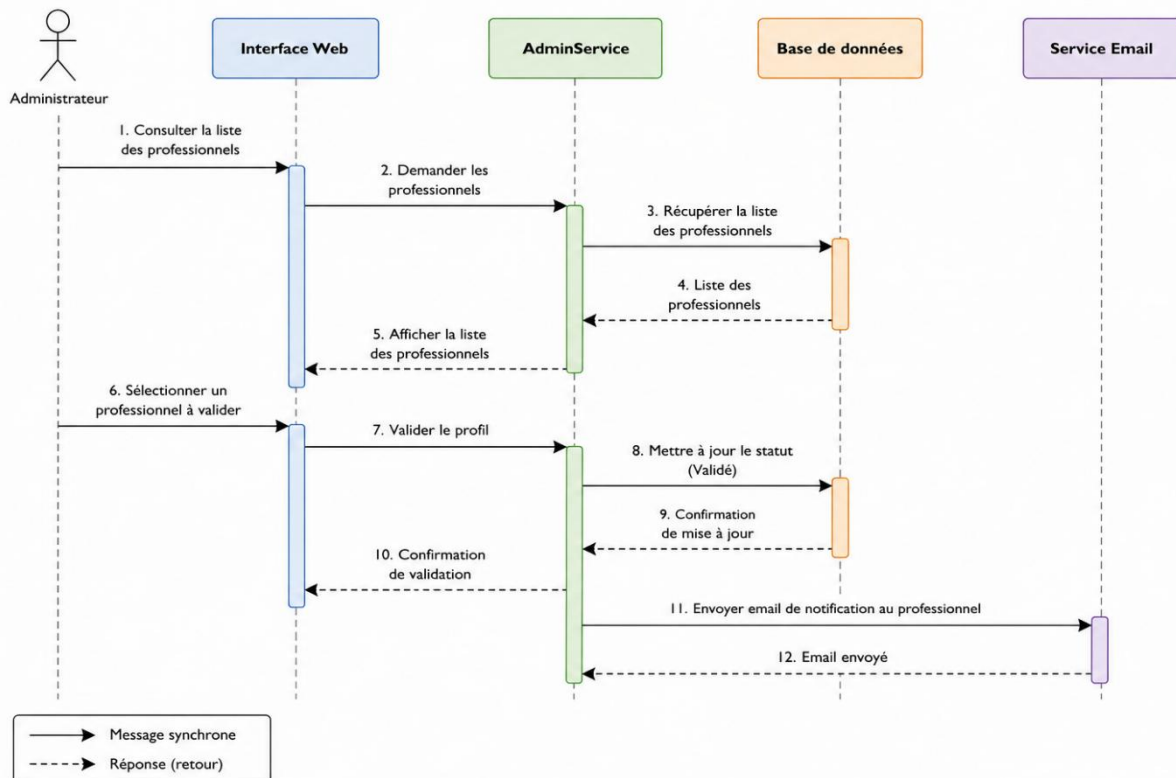


Diagramme de séquence 6.10 — Validation d'un professionnel (Administrateur)

6.1 Description du scénario

- Connexion : le patient saisit ses identifiants, vérifiés par le service d'authentification auprès de la base de données.
- Choix du professionnel et consultation : le patient consulte un profil et demande la liste des créneaux disponibles.
- Réservation du rendez-vous : le service de rendez-vous vérifie la disponibilité en base avant d'enregistrer la réservation.
- Confirmation : un email de confirmation est envoyé au patient et les détails du rendez-vous s'affichent.

7. Architecture technique et choix technologiques

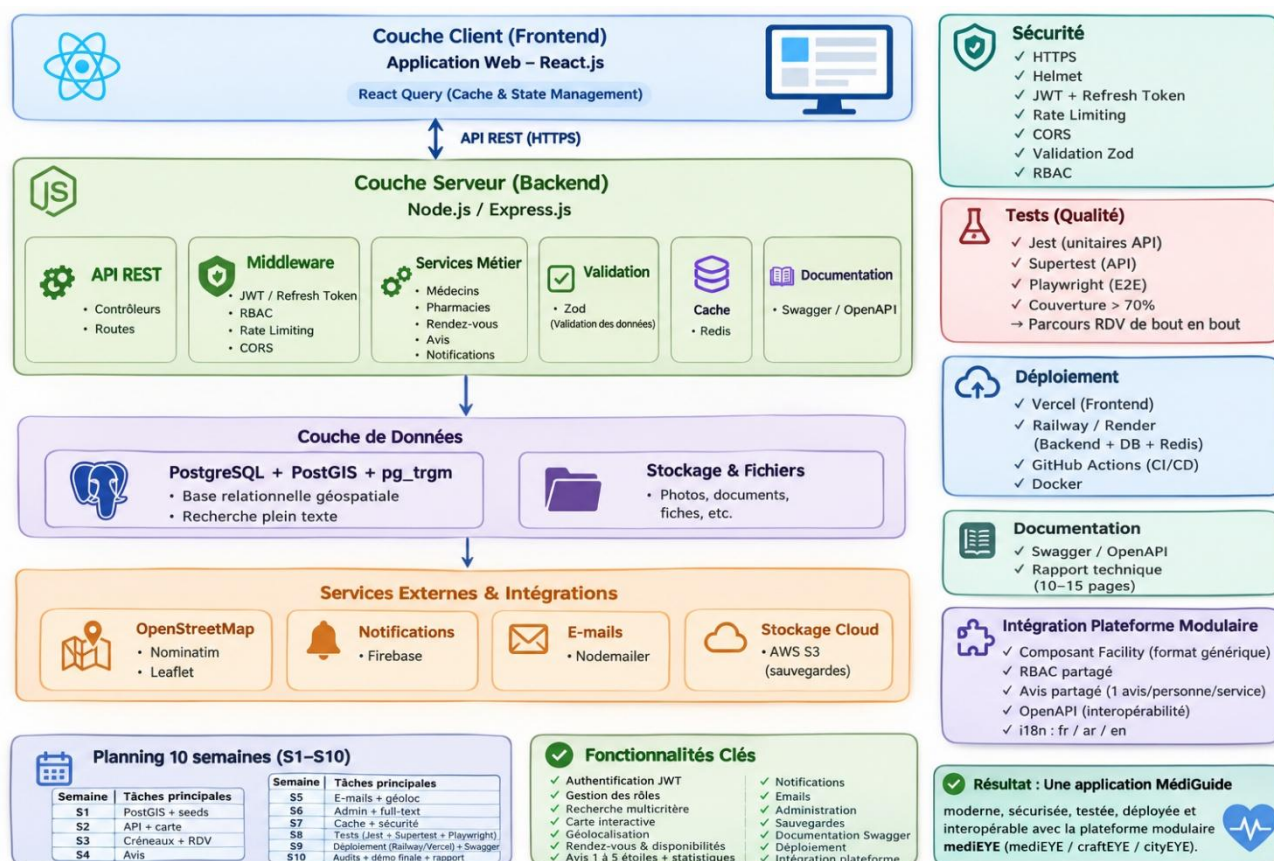


Figure 4 — Architecture technique 3-tiers de MédiGuide

L'application repose sur une architecture 3-tiers : le frontend React communique avec le backend Node.js/Express via une API REST, laquelle interroge PostgreSQL (avec PostGIS et pg_trgm), le cache Redis, et les services externes (Nodemailer, OpenStreetMap/Nominatim/Leaflet).

7.1 Stratégie de tests

- Tests unitaires de l'API avec Jest + Supertest, à chaque sprint (couverture cible > 70 %).
- Tests de composants frontend.
- Tests end-to-end avec Playwright sur le parcours critique « prise de rendez-vous de bout en bout » (sprint S8).

7.2 Planning sur 10 semaines (3 développeurs)

Répartition : dev A (backend), dev B (frontend), dev C (full-stack).

Semaine	Contenu
S1	PostGIS + seeds de données
S2	API REST + carte interactive
S3	Créneaux (Disponibilité) + rendez-vous
S4	Avis (notation 1-5, moyenne SQL)

S5	Emails (Nodemailer) + géolocalisation (Autour de moi)
S6	Espace admin + recherche full-text (pg_trgm)
S7	Cache (Redis) + sécurité (Zod, rate limiting, CORS, refresh)
S8	Tests (Jest, Supertest, Playwright)
S9	Déploiement (Railway/Vercel) + documentation Swagger
S10	Audits, corrections finales, démo

7.3 Déploiement et livrables

- Déploiement : Vercel (frontend) + Railway ou Render (backend, base de données, Redis), en tiers gratuits.
- CI/CD : GitHub Actions (sprint S9).
- Documentation technique de l'API : Swagger / OpenAPI.
- Rapport technique de 10 à 15 pages.
- Soutenance de 20 minutes avec démonstration.

7.4 Intégration à la plateforme modulaire (mediEYE / craftEYE / cityEYE)

MédiGuide n'est pas une application isolée : il s'intègre dans une plateforme modulaire partageant le backend, la carte, la liste, la fiche et la table d'avis avec les autres modules. Les points suivants doivent être anticipés :

- **Format générique Facility** : mapper les entités locales (médecins, pharmacies, parapharmacies) vers le format partagé {id, name, category, subSpecialty, address, phone, latitude, longitude, features, isVerified, distanceM}.
- **Table d'avis partagée** : réutiliser la table d'avis polymorphique commune (1 avis par personne et par service, révisable).
- **Registre RBAC** : enregistrer le module avec le slug « medieye » et les permissions read / write / admin.
- **Routes front** : exposer les vues sous /modules/medieye/* avec le shell commun de la plateforme.
- **Documentation** : fournir un fragment OpenAPI et un script de seed CSV pour le module.
- **Internationalisation** : prise en charge fr / ar / en.

7.5 Tableau récapitulatif des technologies

Composant	Technologie	Justification
Frontend	React 18 + Vite, Leaflet.js, React Query, Tailwind CSS	Interface dynamique et modulaire, cache des requêtes, carte interactive, style responsive.
Backend	Node.js + Express, JWT, Nodemailer	API REST performante, authentification sécurisée, envoi d'emails de confirmation/rappel.
Base de données	PostgreSQL 15 + PostGIS + pg_trgm	Transactions fiables, recherche géospatiale performante (index GiST), recherche full-text/floue.
Cache & sessions	Redis	Cache des recherches géo populaires, stockage des sessions JWT.
Géolocalisation	OpenStreetMap (Nominatim, Leaflet)	Cartographie et géocodage libres et gratuits, adaptés à un projet académique.
Déploiement	Vercel + Railway/Render, GitHub Actions	Hébergement gratuit, intégration continue.